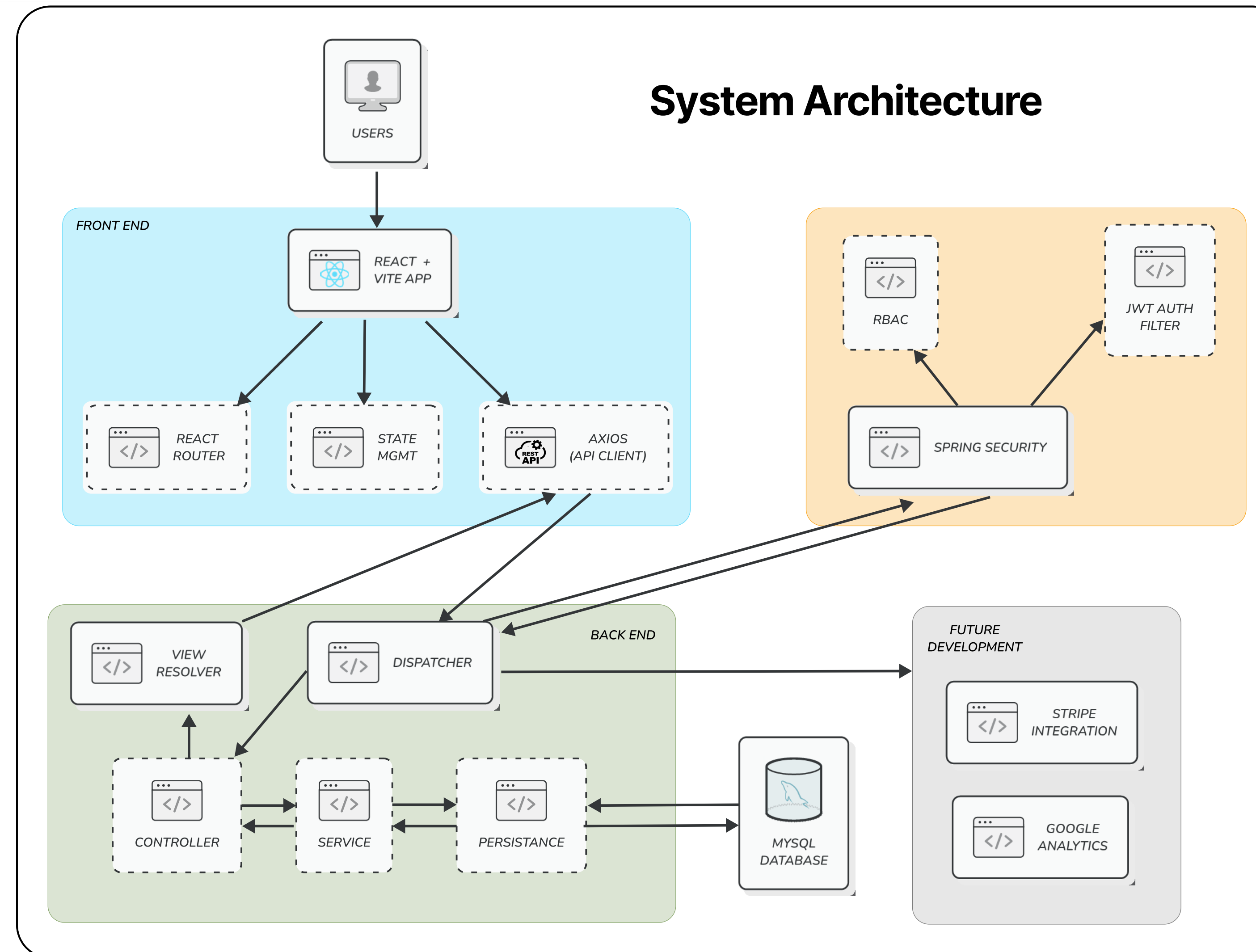


Overview

/dev/depot is a full-stack e-commerce platform for browsing, managing, and purchasing PC components, built using Java Spring Boot, React.js, and MySQL. The backend exposes a RESTful API with JWT-based authentication, role-based access control, and integration with Spring Security and Spring Data JPA. The React frontend features modular components, dynamic routing, and persistent state management via React Context.

Designed with scalability and maintainability in mind, the system follows modern development practices including layered architecture, stateless communication, and separation of concerns. Development was conducted using Agile workflows with version control, iterative testing, and feature-driven deployment. Future iterations aim to incorporate Docker-based deployment and payment gateway integration.

System Architecture



Outcomes

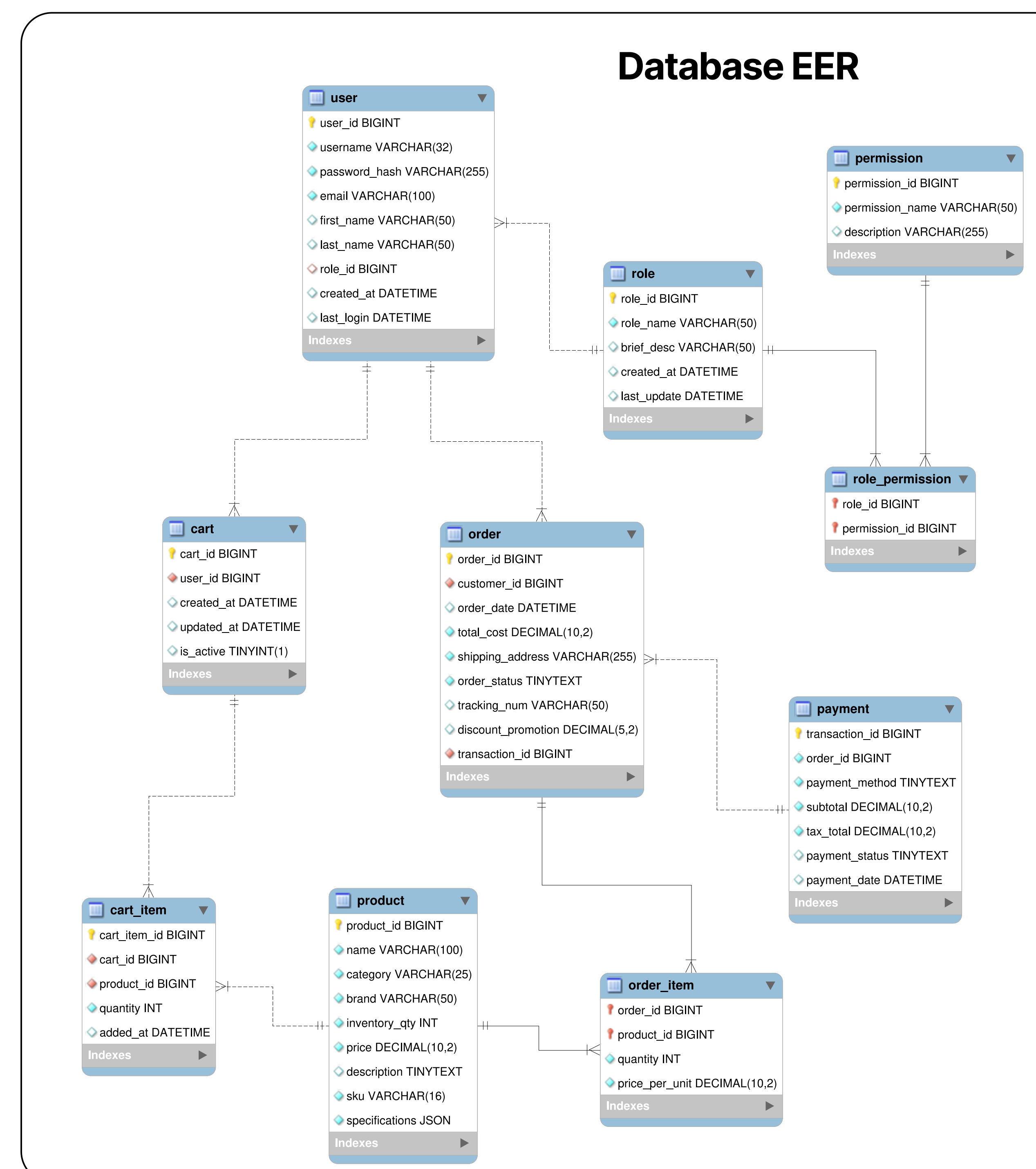
The /dev/depot project culminated in a modular, full-stack e-commerce platform capable of supporting account registration, authentication, product browsing, and cart management for PC components. The backend API, developed with Spring Boot and secured via JWT-based authentication using Spring Security, was designed for stateless communication and role-based access control. On the frontend, React components were implemented with dynamic routing and persistent state managed through React Context. The team adopted agile workflows, using version control, sprint-based planning, and feature-driven development. This foundation enabled clear task ownership and iterative testing throughout the project lifecycle. The final architecture supports both scalability and extensibility, demonstrating the team's application of modern software engineering practices.

System Architecture

/dev/depot is a full-stack e-commerce platform for browsing, managing, and purchasing PC components, built using Java Spring Boot, React.js, and MySQL. The backend exposes a RESTful API with JWT-based authentication, role-based access control, and integration with Spring Security and Spring Data JPA. The React frontend features modular components, dynamic routing, and persistent state management via React Context.

Designed with scalability and maintainability in mind, the system follows modern development practices including layered architecture, stateless communication, and separation of concerns. Development was conducted using Agile workflows with version control, iterative testing, and feature-driven deployment. Future iterations aim to incorporate Docker-based deployment and payment gateway integration.

Database EER



Challenges

One of the most significant technical hurdles was integrating Spring Security with stateless JWT authorization, particularly configuring token lifecycle management. Most members had minimal experience with our stack before starting, which brought along the issue of efficient knowledge sharing. Overall, the challenges surrounding data integrity and formatting caused friction throughout the project. Project velocity was further impacted by uneven contribution within the team, requiring late-stage realignment of responsibilities and additional work from core contributors to ensure delivery.

Future Work

Planned improvements to /dev/depot include integration with third-party payment providers, such as Stripe or PayPal, and the introduction of a fully-featured administrative dashboard for inventory management and order processing. The team also aims to containerize the full application using Docker, simplifying future deployment and scaling scenarios. Additional improvements to the codebase will involve enhancing test coverage, and further decoupling business logic to support microservice-based architectures.